

What are Transformations in Spark?

Transformations are operations on an RDD/DataFrame that **don't execute immediately**. They build a **logical plan (DAG)**.

Example:

```
rdd2 = rdd1.map(lambda x: x * 2)
```

No execution yet — Spark is just planning.

♦ 2. Narrow vs Wide Transformations (Core Idea)

✅ Narrow Transformation

A transformation is *narrow* when:

Each output partition depends on **only one input partition**

- 👉 No data movement across partitions
- 👉 No shuffle
- 👉 Fast, pipeline-friendly

♦ Examples:

- `map()`
- `filter()`
- `flatMap()`
- `mapPartitions()`

♦ Visualization:

Partition 1 → Partition 1

Partition 2 → Partition 2

Partition 3 → Partition 3

- 👉 Data stays **local**
-

✅ Wide Transformation

A transformation is *wide* when:

Output partition depends on **multiple input partitions**

- 👉 Requires **data shuffle across network**
- 👉 Expensive operation
- 👉 Breaks pipeline into stages

♦ **Examples:**

- `groupByKey()`
- `reduceByKey()`
- `join()`
- `distinct()`
- `repartition()`

♦ **Visualization:**

Partition 1 → Partition A

Partition 2 → Partition A

Partition 3 → Partition B

👉 Data is **redistributed across cluster**

🔥 **3. The REAL Difference (Senior-Level Insight)**

Feature	Narrow	Wide
Data movement	❌ No shuffle	✅ Shuffle
Execution	Single stage	Multiple stages
Performance	Fast	Slow
Fault recovery	Easy	Complex
Pipeline	Can be pipelined	Breaks pipeline

🔥 **4. How Spark Executes Internally**

Spark builds a **DAG (Directed Acyclic Graph)**

👉 Narrow transformations are grouped into **one stage**

👉 Wide transformations create **stage boundaries**

Example:

```
rdd.map().filter().map().reduceByKey()
```

Execution:

Stage 1: `map` → `filter` → `map` (all narrow)

Stage 2: `reduceByKey` (wide → shuffle happens here)

👉 This is called **stage splitting due to shuffle**

🔥 5. Practical Scenario (Real Data Engineering Use Case)

🧠 Scenario: Log Processing Pipeline

You have web logs:

(user_id, page, time)

Goal:

Find **total visits per user**

❌ Bad Approach (costly):

```
rdd.map(lambda x: (x.user_id, 1)) \
    .groupByKey() \
    .mapValues(len)
```

Problem:

- groupByKey() → Wide transformation
 - Moves **ALL data across network**
 - High memory + slow
-

✅ Optimized Approach:

```
rdd.map(lambda x: (x.user_id, 1)) \
    .reduceByKey(lambda a, b: a + b)
```

Why better?

- reduceByKey() does **local aggregation first (map-side combine)**
 - Less data shuffled
 - Faster and scalable
-

🔥 6. Another Real Scenario: Join Optimization

❌ Problem:

Joining two large datasets

```
df1.join(df2, "user_id")
```

👉 Causes **wide transformation + massive shuffle**

✅ Optimization Strategy:

- Use **broadcast join** if one dataset is small

from pyspark.sql.functions import broadcast

df1.join(broadcast(df2), "user_id")

👉 Avoids shuffle → converts wide → almost narrow behavior

🔥 7. Key Interview Insight (Very Important)

👉 Not all wide transformations are bad

👉 But **you must minimize shuffles**

Golden Rule:

"Shuffle is the most expensive operation in Spark"

🔥 8. How to Identify in Interview Quickly

Ask yourself:

- Does data move across partitions? → Wide
 - Does each partition work independently? → Narrow
-

🔥 9. Final Mental Model

Think like this:

👉 Narrow = **"Work locally"**

👉 Wide = **"Redistribute globally"**

🔥 10. One-Line Summary (Interview Ready)

Narrow transformations operate on a single partition without data movement, while wide transformations require shuffling data across partitions, creating stage boundaries and impacting performance.

◆ Big Picture First

When you run Spark code, it doesn't just "run".

It breaks your work into **3 levels**:

Job → Stage → Task

Think of it like a company:

- **Job** = Full project
- **Stage** = Phase of the project
- **Task** = Individual worker doing small work

🔥 1. What is a JOB?

👉 A **Job** is created when you run an **action**

Examples of actions:

- `count()`
- `collect()`
- `save()`

Example:

```
rdd.map(...).filter(...).count()
```

👉 `count()` triggers **1 Job**

Simple Definition:

A **Job** = Entire work Spark does for one action

🔥 2. What is a STAGE?

👉 A **Stage** is a group of operations that can run **without shuffle**

👉 Spark splits stages when it sees a **wide transformation**

Example:

```
rdd.map().filter().reduceByKey()
```

Breakdown:

Stage 1 → map + filter (no shuffle)

Stage 2 → reduceByKey (shuffle happens)

Simple Definition:

A **Stage** = Set of transformations that run together (no data movement)

🔥 3. What is a TASK?

👉 A **Task** is the **smallest unit of work**

👉 Each task works on **one partition**

Example:

If you have:

- 3 partitions

Then:

Stage → 3 Tasks

Task 1 → Partition 1

Task 2 → Partition 2

Task 3 → Partition 3

Simple Definition:

A **Task** = Work done on one partition by one CPU core

🔥 4. Full Flow (Very Important)

Code:

```
rdd.map().filter().reduceByKey().count()
```

Execution:

Action → count()

➡ Job 1 created

➡ Stage 1:

map + filter

→ 3 tasks (if 3 partitions)

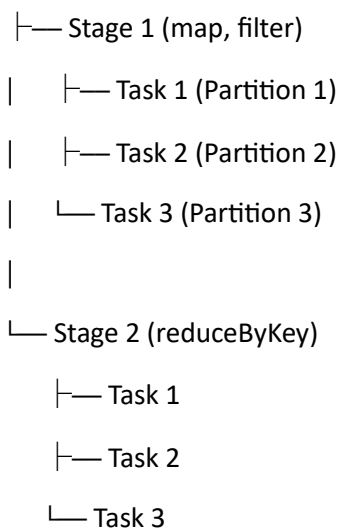
➡ Stage 2:

reduceByKey (shuffle)

→ 3 tasks

🔥 5. Visual Flow

Job



🔥 6. Real-Life Analogy (Easy to Remember)

Imagine food delivery:

- **Job** → Deliver full order
- **Stage** → Cooking → Packing → Delivery phases
- **Task** → Each chef cooking one dish

🔥 7. Key Differences (Super Simple Table)

Concept	Meaning	Trigger
Job	Full execution	Action
Stage	Group of ops	Shuffle boundary

Concept	Meaning	Trigger
Task	Work unit	Partition

🔥 8. Interview One-Liner (Perfect Answer)

“A job is triggered by an action, stages are created by splitting the job at shuffle boundaries, and tasks are the smallest units of execution that run on partitions in parallel.”

🔥 9. One Important Insight (Senior Level)

- 👉 Increasing partitions = increasing tasks
- 👉 More tasks = more parallelism (but overhead too)

**LINKING TRANSFORMATIONS → SHUFFLE
→ DAG → STAGES → TASKS → CLUSTER
BEHAVIOR.**

◆ 1. Narrow vs Wide Transformations (Core Idea)

✅ Narrow Transformation (No Shuffle)

- 👉 Each output partition depends on **only one input partition**

Examples:

- map()
- filter()
- flatMap()

💡 What actually happens in cluster

- Data stays **on the same executor**
- No network transfer
- Runs as a **pipeline (single stage)**

Mental Model

Partition P1 → P1'

Partition P2 → P2'

Partition P3 → P3'

 No mixing of data across nodes

Wide Transformation (Shuffle Happens)

 Output partition depends on **multiple input partitions**

Examples:

- `reduceByKey()`
- `groupByKey()`
- `join()`

What happens internally

- Data is **re-distributed across executors**
- Requires **network I/O**
- Creates **shuffle boundary**
- Triggers **new stage**

Mental Model

P1 ----\

P2 ----- → Shuffle → New partitions → Aggregation

P3 ----/

Why Shuffle is Expensive

From your slide: *“Wide transformations involve shuffle which is costly”*

Real reasons:

1. Disk I/O (spill to disk)
2. Network transfer
3. Serialization/deserialization
4. Data skew issues
5. Memory pressure

👉 That's why:

Always minimize wide transformations or push them to the end

◆ 2. Your Diagram Explained (Step-by-Step Execution)

Step 1: Input File (e.g., 500MB)

- Split into partitions → P1, P2, P3, P4
 - Sent to different worker nodes
-

Step 2: Narrow Transformation (map)

- Runs **locally inside each worker**
- No shuffle

Worker 1: P1 → R1

Worker 2: P2 → R2

Worker 3: P3 → R3

Worker 4: P4 → R4

👉 This is **local aggregation phase**

Step 3: Wide Transformation (reduceByKey)

Now comes the important part:

🔄 Shuffle Phase

- Keys are **repartitioned**
- Same keys go to same node

Example:

Key A → Node 1

Key B → Node 2

Key C → Node 3

👉 Data moves across network

Step 4: Reducer (Global Aggregation)

- Final aggregation happens
- Output is produced

👉 This is **global aggregation**

◆ 3. DAG (Directed Acyclic Graph)

Spark builds a **logical plan (DAG)** of transformations.

From your image:

- `textFile` → `map` → `reduceByKey`

DAG View

`textFile` → `map` → `reduceByKey` → `collect`

◆ 4. Job, Stage, Task (MOST IMPORTANT 🔥)

✅ Job

- Triggered by an **action**

Examples:

- `collect()`
- `count()`
- `save()`

👉 1 action = 1 job

✅ Stage

From your note:

“Number of stages depends on number of wide transformations”

Rule:

Stages = Number of shuffle boundaries + 1

Example:

`map` → `filter` → `reduceByKey` → `map` → `collect`

👉 Stages:

1. Stage 1 → `map` + `filter` (narrow)
 2. Stage 2 → `reduceByKey` (shuffle boundary)
 3. Stage 3 → final `map`
-

✅ Task

From your slide:

“Number of tasks = number of partitions”

👉 Each partition = 1 task

Example:

- 4 partitions → 4 tasks per stage
-

◆ 5. Why 2 Stages in Your Image?

Because:

- `reduceByKey()` is a **wide transformation**

Breakdown:

Stage 0:

textFile → map (no shuffle)

Stage 1:

`reduceByKey` → `mapPartitions` (after shuffle)

👉 Shuffle splits execution into stages

◆ 6. Important Hidden Insight (Senior-Level)

🔥 1. Shuffle Writes to Disk

Your note:

“result of stage0 will go to disk”

✓ Correct:

- Shuffle data is written to **local disk**
 - Next stage reads from disk
-

🔥 2. Pipeline Optimization

Spark **pipelines narrow transformations**

map → filter → flatMap

👉 Runs in **one stage**, no intermediate storage

🔥 3. reduceByKey vs groupByKey

Operation	Behavior	Efficiency
-----------	----------	------------

groupByKey	shuffle all data	❌ bad
------------	------------------	-------

reduceByKey	local + shuffle	✅ better
-------------	-----------------	----------

👉 Always prefer reduceByKey

🔥 4. Data Skew Problem

If one key is huge:

Key A → 90% data

👉 One node becomes bottleneck

Solution:

- Salting
 - Custom partitioner
-

🔥 5. Partitioning Matters

- More partitions → more parallelism
- Too many → overhead

Rule of thumb:

2–4 tasks per CPU core

💠 7. Real Cluster Execution (Putting Everything Together)

Example Code:

```
rdd = sc.textFile("data.txt")
rdd2 = rdd.map(lambda x: (x,1))
rdd3 = rdd2.reduceByKey(lambda a,b: a+b)
rdd3.collect()
```

Execution Flow:

Job Created:

- Due to collect()
-

Stage 1 (No Shuffle)

- Tasks = partitions
 - Operations:
 - read file
 - map
-

Stage 2 (Shuffle Stage)

- Shuffle write
 - Shuffle read
 - reduceByKey aggregation
-

Final Output:

- Returned to driver
-

◆ 8. Best Practices (Interview + Real World)

- ✓ Push filters early
 - ✓ Reduce data before shuffle
 - ✓ Avoid unnecessary wide transformations
 - ✓ Use reduceByKey instead of groupByKey
 - ✓ Tune partitions (repartition, coalesce)
 - ✓ Watch Spark UI (DAG & stages)
-

← END Final Simple Summary

- Narrow → no shuffle → fast → same node
- Wide → shuffle → slow → data movement
- Action → Job
- Shuffle → New Stage
- Partition → Task